# Day 18: Wazuh Component Health Check

*Status Report and Planned Verification Procedure*

SOC Internship - TriosCyber

| Field | Details |
|---|---|
| Intern Name | Amrita Pathak |
| Date | 20 September 2026 |
| Internship Track | SOC (Security Operations Center) |
| Assigned Task | Verify Wazuh server components are running and accessible; record service status, dashboard access, and any installation troubleshooting performed (Tools: systemctl, Wazuh Dashboard). |
| Status of this Report | **Interim status report. The practical health check has NOT yet been performed, because it depends on a working Wazuh installation which is still in progress (see Day 17). This report honestly documents what has actually been completed today, and sets out the verification procedure planned for execution once installation succeeds.** |

# Table of Contents

Table of Contents ..... 2
Key Terms ..... 3
Important Note on This Report ..... 3
Executive Summary ..... 3
1. Objective ..... 5
2. Work Completed Today (Real, Verified) ..... 5
3. Remaining Steps to Reach a Working Installation (Planned, Not Yet Executed) ..... 5
4. Planned Health Check Procedure (To Be Executed in Phase E) ..... 7
4.1 Service status verification via systemctl ..... 7
4.2 Dashboard accessibility verification ..... 7
4.3 What will be recorded as evidence ..... 7
5. Conclusion ..... 7

## Key Terms

- **systemctl:** Linux command used to start, stop, and check the status of systemd-managed services; used here to verify each Wazuh component is running (active) or not.
- **Wazuh Dashboard:** The web-based interface (built on OpenSearch Dashboards) used to view alerts, search log data, and confirm the Wazuh stack is reachable and functional.
- **wazuh-indexer:** The OpenSearch-based component that stores and indexes all security event data; the component that failed to start in every Day 17 attempt.
- **wazuh-manager:** The core Wazuh server process that receives, analyzes, and correlates data from agents and log sources.
- **Filebeat:** The lightweight shipper that forwards data from the Wazuh manager into the Wazuh indexer for storage.
- **JVM Heap:** The block of memory the Java Virtual Machine (which OpenSearch/the Wazuh indexer runs on) reserves for itself; Wazuh's official guidance is to set it explicitly to half of total system RAM rather than letting it auto-size, which was the root cause identified in Day 17.
- **Dual Boot:** A configuration where two operating systems (here, Windows and Ubuntu Server) are installed on the same physical machine, with the user choosing which one to start at power-on; only one runs at a time.
- **Unallocated Space:** Disk space that has been freed (for example, by shrinking an existing Windows partition) but not yet assigned to any partition or filesystem; this is where the new Ubuntu installation will be created.
- **EFI System Partition (ESP):** A small partition required for UEFI-based booting that can hold boot files for multiple operating systems; the plan is to reuse Windows' existing ESP rather than create a conflicting second one.
- **journalctl:** Linux command used to view detailed systemd service logs; used in Day 17 to reveal the exact reason (a start-operation timeout) behind the wazuh-indexer failures, and planned for use again if any component fails its health check.

## Important Note on This Report

**This report is submitted honestly as an interim status update, not as a record of a completed health check. No Wazuh component was running on any machine at the time of writing, so no systemctl output or dashboard screenshot in the strict Day 18 sense exists yet to record. Rather than fabricate output that was never actually observed, this report documents (a) the real, verifiable work completed today toward getting Wazuh running, and (b) the exact verification procedure that will be executed and documented with real evidence once installation succeeds, expected within the next session.**

## Executive Summary

Following Day 17's finding that Wazuh could not be reliably started inside a VMware-virtualized Ubuntu guest on a memory-constrained host (root cause: JVM heap auto-sizing under memory/CPU contention introduced by the hypervisor layer), the decision was made to move the Wazuh deployment to a native dual-boot Ubuntu Server installation on the desktop host, removing the virtualization layer entirely rather than continuing to work around it.

Today's session began this transition. The Windows D: drive on the target desktop was backed up and then shrunk by 100GB to create unallocated space for the new Ubuntu installation, using Windows Disk Management (Shrink Volume). This step completed successfully and was verified directly: Disk Management now shows a 100GB unallocated region adjacent to D:, with no data loss to the existing partition. The remainder of the migration, installing Ubuntu Server into that space as a dual-boot system, installing Wazuh with a correctly-sized JVM heap set before first service start, and then performing the Day 18 health check itself, was not completed in today's session and is planned for the next session.

**This report therefore covers two things honestly and separately: the real disk-preparation work completed today (Section 2), and the planned, not-yet-executed procedure for completing installation and then performing the actual Day 18 health check (Sections 3 and 4), so the verification methodology is documented and ready to execute as soon as Wazuh is running.**

## 1. Objective

Verify that all Wazuh server components (indexer, manager, dashboard, and the Filebeat shipper) are running correctly after installation, using systemctl to check service status and the Wazuh Dashboard web interface to confirm the stack is reachable and functional, recording real command output and screenshots as evidence.

## 2. Work Completed Today (Real, Verified)

The following was actually performed on the target desktop (Dell PC, i5, 8GB RAM, 1TB HDD) today and is reported factually.

| Step | Action | Result |
|---|---|---|
| 1 | Backed up existing data from the desktop's D: drive prior to any partition changes. | Completed. |
| 2 | Opened Windows Disk Management (diskmgmt.msc), right-clicked the D: volume, selected Shrink Volume, and shrank it by 102400MB (100GB). | Completed and verified. |
| 3 | Confirmed a new 100GB Unallocated region now sits adjacent to D:, with D:'s existing files untouched (Shrink Volume does not move or delete existing data, it only reclaims already-free space). | Verified in Disk Management. |

This 100GB region is reserved for the Ubuntu Server installation described in Section 3, and is not yet formatted or partitioned further.

## 3. Remaining Steps to Reach a Working Installation (Planned, Not Yet Executed)

The following phases are planned for the next session, in order. None of these have been executed yet.

| Phase | Description | Status |
|---|---|---|
| **A** | Back up critical data and shrink Windows D: drive to free 100GB of unallocated space. | **Complete** |
| **B** | Download Ubuntu Server 24.04 LTS ISO and create a bootable USB installer using Rufus. | **Planned** |
| **C** | Boot from USB, use the installer's custom/manual storage layout to install Ubuntu Server into the 100GB unallocated space, reusing (not reformatting) the existing Windows EFI System Partition so both operating systems remain bootable via GRUB. | **Planned** |
| **D** | Install Wazuh (all-in-one) on the native Ubuntu installation. Before starting the wazuh-indexer service | **Planned** |

| | | |
|---|---|---|
| | for the first time, set -Xms4g and -Xmx4g in /etc/wazuh-indexer/jvm.options (per Wazuh's official indexer tuning guidance: heap should equal half of total system RAM, with min and max matched exactly), rather than relying on the installer's default auto-sizing, which was identified in Day 17 as the root cause of repeated failure under virtualization. | |
| **E** | Perform the actual Day 18 health check described in Section 4 below, with real command output and a real dashboard screenshot. | **Planned** |

## 4. Planned Health Check Procedure (To Be Executed in Phase E)

This section documents the exact verification procedure that will be followed once Wazuh is installed and running, to demonstrate understanding of the Day 18 task ahead of its practical execution.

### 4.1 Service status verification via systemctl

Each Wazuh component runs as its own systemd service. The following commands will be run, and a healthy stack should report active (running) for all four:

```
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-dashboard
sudo systemctl status filebeat
```

- wazuh-indexer: the OpenSearch-based data store. A failed or inactive state here (as seen repeatedly in Day 17) blocks the rest of the stack, since the manager and dashboard both depend on it.
- wazuh-manager: the core Wazuh server process that receives, analyzes, and correlates agent and log data.
- wazuh-dashboard: the OpenSearch Dashboards-based web interface used to view alerts and data.
- filebeat: ships data from the manager into the indexer; expected to be active alongside the other three.

For any service not reporting active (running), the plan is to immediately capture its detailed log with journalctl (for example, sudo journalctl -u wazuh-indexer --no-pager | tail -50, the same diagnostic approach already used successfully in Day 17 to surface the systemd timeout evidence) rather than treating a status check as pass/fail with no explanation.

### 4.2 Dashboard accessibility verification

Once all four services report active, the plan is to open a browser and navigate to https://<server-ip> (or https://localhost if accessed directly on the machine), accept the self-signed certificate warning, and log in using the admin credentials generated in wazuh-passwords.txt during installation. A successful login and a populated dashboard homepage will be captured as a screenshot for the real Day 18 submission.

### 4.3 What will be recorded as evidence

- Full terminal output of all four systemctl status commands.
- Any journalctl output used to diagnose a service that is not healthy on first check, and the corrective action taken, consistent with how Day 17's troubleshooting was documented.
- A screenshot of the Wazuh Dashboard login screen and, if login succeeds, the dashboard homepage.

## 5. Conclusion

The practical Day 18 health check has not yet been performed, and this report does not claim otherwise. What has been completed, and is reported honestly, is the disk preparation step (100GB freed on the target desktop) needed to move Wazuh off the resource-constrained VM setup diagnosed in Day 17 and onto a native, dual-boot installation. The remaining installation phases and the health check itself are planned in detail above and will be carried out and documented with real command output and screenshots in the next session, at which point a verified Day 18 report will follow this one.